Remote Web Based Control – Web Server

In the last Lap we have seen the design and development of an interesting Wi-Fi application which gets the ambient temperature and humidity and stores on the cloud.

In this Lap we will be developing another interesting application. Here, the system will control remotely two LEDs connected to an ESP32 DevKitC development board using an HTTP Web Server application. Although in this example two LEDs are used, relays can generally be connected to DevKitC and any type of electrical equipment can be controlled remotely.

The Block Diagram

The block diagram of the system is shown in Figure 9.1. Figure 9.2 shows the circuit diagram of the project. LED0 and LED1 are connected to GPIO ports 23 (LED0) and 22 (LED1) through 330-ohm current limiting resistors. The LEDs are turned ON when the port output is at logic 1 (high) and turned OFF when the port output is at logic 0 (low).

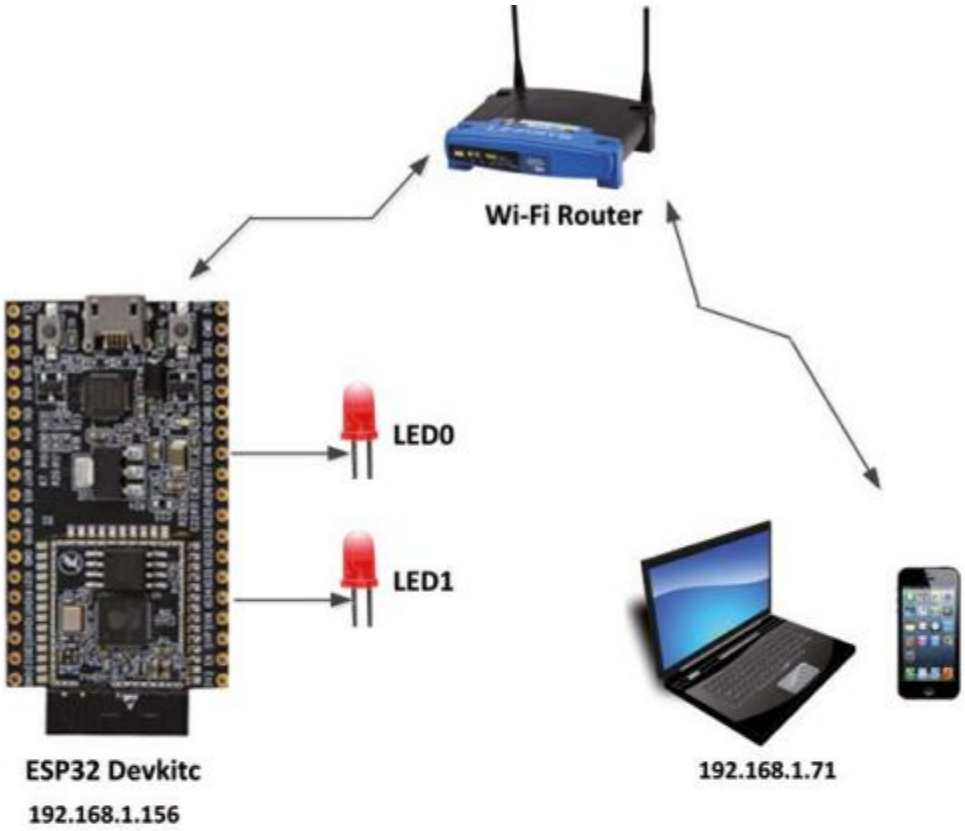

Figure 9.1 Block diagram of the system

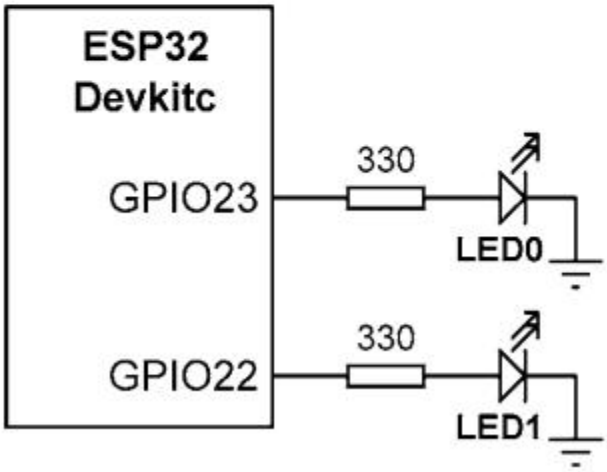

Figure 9.2 Circuit diagram of the system

HTTP Web Server/Client

A web server is a program that uses HTTP (Hypertext Transfer Protocol) to serve web pages to users in response of their requests. These requests are forwarded by HTTP clients. By using the HTTP server/client pair we can control any device connected to a web server processor over the web.

Figure 9.3 shows the structure of a web server/client setup. In this figure, the ESP32 DevKitC is the web server and the PC (or a laptop, tablet or a mobile phone) is the web client. The device to be controlled is connected to the web server processor. In this example we have two LEDs connected to the ESP32 DevKitC development board. The operation of the system is as follows:

- The web server is in the listen mode, listening for requests from the web client
- The web client makes a request to the web server by sending an HTTP request
- In response, the web server sends an HTTP code to the web client which is activated by the web browser by the user on the web client and is shown as a form on the web client screen.
- The user sends a command (e.g. ticks a button on the web client form to turn ON an LED) and this sends a code to the web server so that the web server can carry out the required operation.

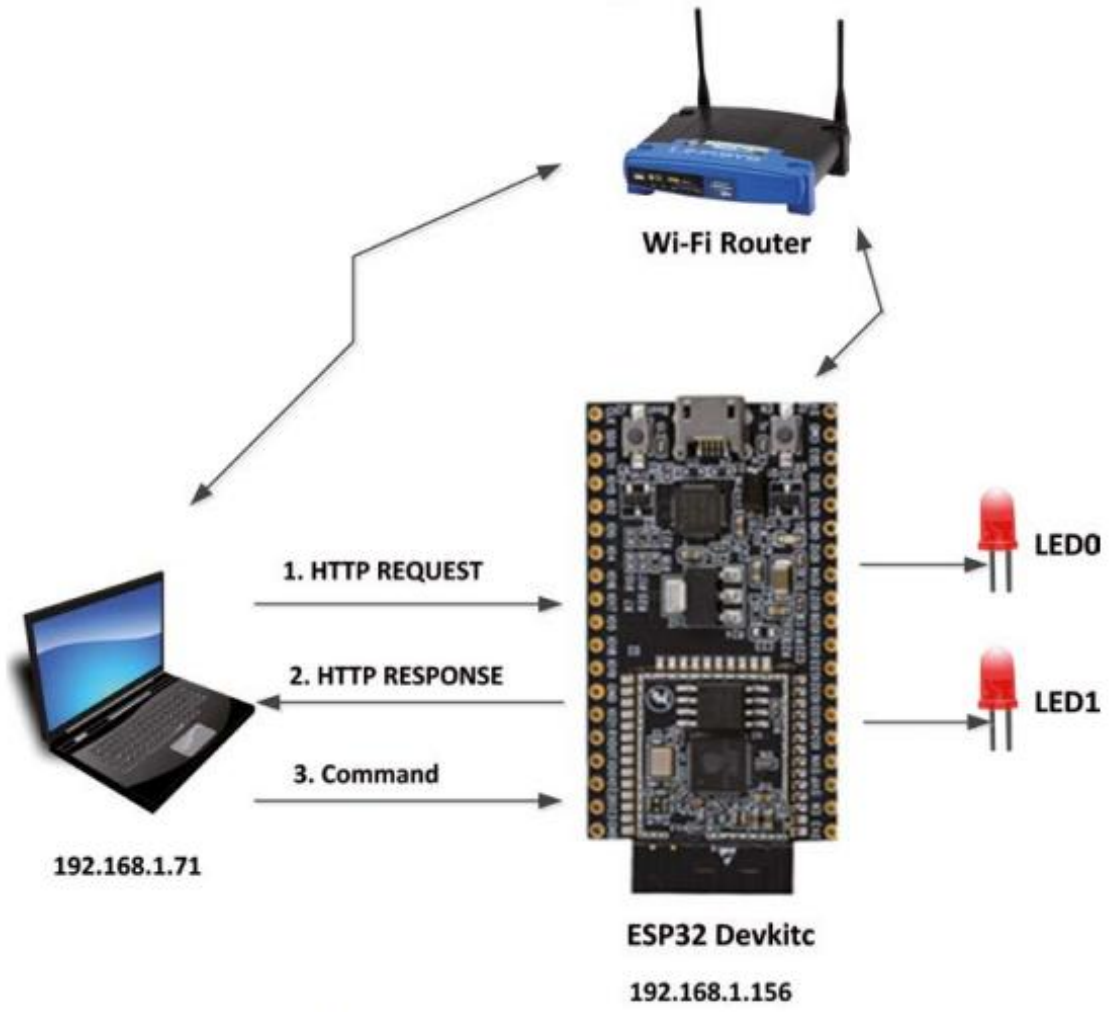

Figure 9.3 Web server/client structure

In this program a laptop with IP address 192.168.1.71 is used as the web client and the ESP32 DevKitC with the IP address 192.168.1.156 is used as the web server

ESP32 DevKitC Program Listing

Figure 9.4 shows the complete program (program: WEBSERVER). At the beginning of the program, the Wi-Fi library is included in the program, LED0 and LED1 are assigned to GPIO ports 23 and 22 respectively, and the local Wi-Fi name and password are defined.

The program then defines the HTML code that is to be sent to the web client when a request comes from it. This code is stored in variable html. When executed by the web browser (e.g. Internet Explorer) on the web client, the display in Figure 9.5 is shown on the client screen.

A heading is displayed at the centre of the screen and buttons are displayed at the left hand side. Two pairs of buttons are displayed: one pair to turn ON/OFF LED0 and another pair to turn ON/OFF LED1. The ON buttons are green, and the OFF buttons are red.

The web server program sends the html file (statement client .print(html)) to the client and waits for a command from the client. When a button is clicked on the client, a command is sent to the web server in addition to some other data. Here, we are only interested in the actual command sent. The following commands are sent to the web server for each button pressed:

Button Pressed	Command sent to web server
LED0 ON	/?LED0=ON
LED0 OFF	/?LED0=OFF
LED1 ON	/?LED1=ON
LED1 OFF	/?LED1=OFF

The web server program on the ESP32 DevKitC makes a TCP connection and reads the command from the web client as a string (String request = client .readStringUntil('\r')). The program then searches the received command to see if any of the above commands are present in the received data. If the sub-string is not found then a -1 is returned. For example, consider the following statement:

```
if(request.indexOf("LED0=ON") != -1)digitalWrite(LED0, HIGH);
```

Here, sub-string **LED0=ON** is searched in string **request**. A -1 is returned if the sub-string **LED0=ON** does not exists in string request. The program looks for a match for all the 4 commands and if a command is found then the corresponding LED is turned ON or OFF accordingly:

```
if(request.indexOf("LED0=ON") != -1)digitalWrite(LED0, HIGH);
if(request.indexOf("LED0=OFF") != -1)digitalWrite(LED0, LOW);
if(request.indexOf("LED1=ON") != -1)digitalWrite(LED1, HIGH);
if(request.indexOf("LED1=OFF") != -1)digitalWrite(LED1, LOW);
```

Figure 9.5 The screen when the HTML code is executed by the client

In this project, the IP address of the web server was 192.168.1.156. The procedure to test the system is as follows:

- Compile, upload and run the program on the ESP32 DevKitC.
- Open your web browser on your PC and enter the web site address 192.168.1.156.
- The form shown in Figure 9.5 will be displayed on your PC screen.
- Click a button, e.g. LED0 ON, to turn ON LED0.

Figure 9.6 and Figure 9.7 show the commands sent to the web server when LED0 ON button is clicked, and also when LED0 OFF button is clicked.

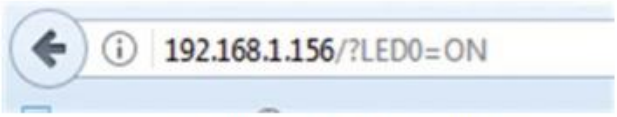

Figure 9.6 Clicking LED0 ON button

Figure 9.7 Clicking LED0 OFF button

Accessing Web Server From Anywhere

In our project the web server can only be accessed locally using the local Wi-Fi router. In this section we will see how to access our web server from anywhere in the world, so that, for example, appliances in our house can be controlled and monitored remotely from anywhere in the world.

We will be using a free service called ngrok in order to create a tunnel to our ESP32 so that we can access it from anywhere in the world. First of all we have to change our default port number from 80 to something else, e.g. 8888, since for some reason ngrok does not work with port number 80. The steps to install and use ngrok are as follows:

- Go to web site https://ngrok.com
- Enter your details as shown in Figure 9.8 to create an account.

Figure 9.8 Enter your details

Click Auth link at the left hand side to get your unique Tunnel token as shown in Figure 9.9. In this example author's tunnel token is: gqudoMhDuJeXnLqVGHqv_7jm25SyUTtbYpnQzTk3Eb

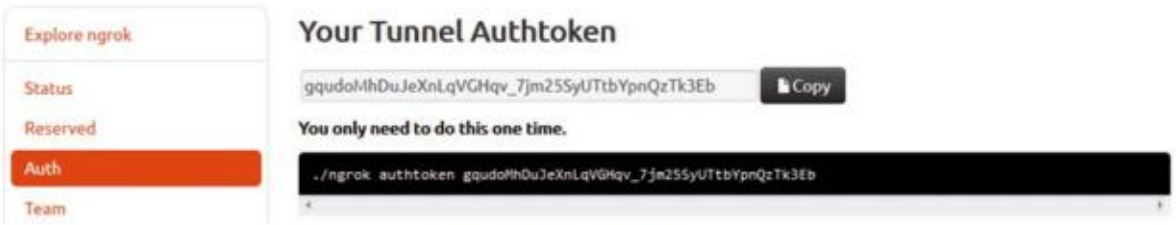

Figure 9.9 Tunnel token

- Click the **Download** tab at the top of the screen and install **ngrok**

- Unzip file to a folder and run the **ngrok.exe** application. You should see a window as in Figure 9.10.

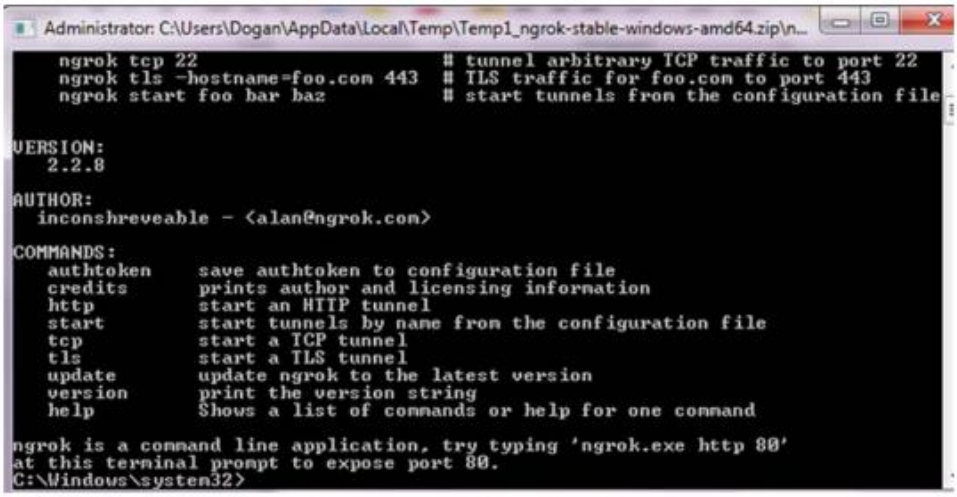

Figure 9.10 Running ngrok

- Enter the following command by replacing the code in < > with your own IP address and **ngrok** Token:

  ngrok tcp <ESP-IP-ADDRESS>:8888 --authtoken <Your ngrok Token>

  For our project in this chapter the command should be:

  ngrok tcp 192.168.1.156:8888 --authtoken gqudoMhDuJeXnLqVGHqv_7jm25SyUTtbYpnQzTk3Eb

  The command should look as shown in Figure 9.11 on the PC terminal.

Figure 9.11 Command on the PC terminal

- You should now see that your tunnel is online and a Forwarding URL will be given as shown in Figure 9.12. In this example the Forwarding URL is: **tcp://0.tcp.ngrok.io:17386**

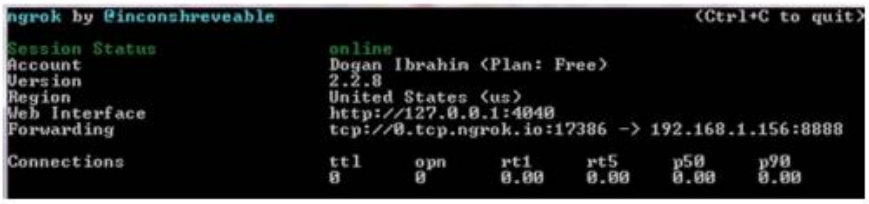

Figure 9.12 The Forwarding URL

In order to access your web server anywhere in the world you should enter your unique URL in a browser, which in this example is:

http://0.tcp.ngrok.io:17386

Notice that your PC must be up and running ngrok while you access your web server. You will be asked to enter your username and password in order to open your web server.